

POSTECH FIAP

Arquitetura e Desenvolvimento Java

RELATORIO TÉCNICO

Tech Challenge - Fase 03

Clinic Scheduler — Sistema Hospitalar de Agendamento, Histórico e Notificações

	Alunos	RM370614 - Vinicius Campos Fanti
		RM373960 - Alex Sousa
		RM373739 - Lucas Gonçalves Prado das Neves
		RM372334 - Higo Otaviano da Rocha
		RM371904 - Marcos Sanches Polido Junior
	Curso	Arquitetura e Desenvolvimento Java
	Tecnologia	Spring Boot 3.5/4.1 + Java 21 + PostgreSQL 17 + RabbitMQ + GraphQL + Docker
	Data	14/09/2026
	Link GitHub	https://github.com/vinifanti/clinic-scheduler-fiap-fase3

Sumário

- 1. Descrição da Arquitetura
- 2. Segurança e Perfis de Acesso
- 3. Modelagem de Domínio
- 4. Endpoints Disponíveis (REST)
- 5. GraphQL — Histórico do Paciente
- 6. Comunicação Assíncrona (RabbitMQ)
- 7. Documentação Swagger / OpenAPI
- 8. Coleção Postman
- 9. Estrutura do Banco de Dados
- 10. Execução com Docker Compose
- 11. Testes Automatizados
- 12. Entregáveis da Fase 3 — Checklist

1. Descrição da Arquitetura

A aplicação foi desenvolvida como um conjunto de microsserviços Spring Boot (Java 21), seguindo arquitetura em camadas dentro de cada serviço (Controller / Service / Repository) e separação de responsabilidades entre serviços por domínio de negócio: agendamento, histórico e notificação, orquestrados por um API Gateway de entrada única.

Diagrama de Fluxo

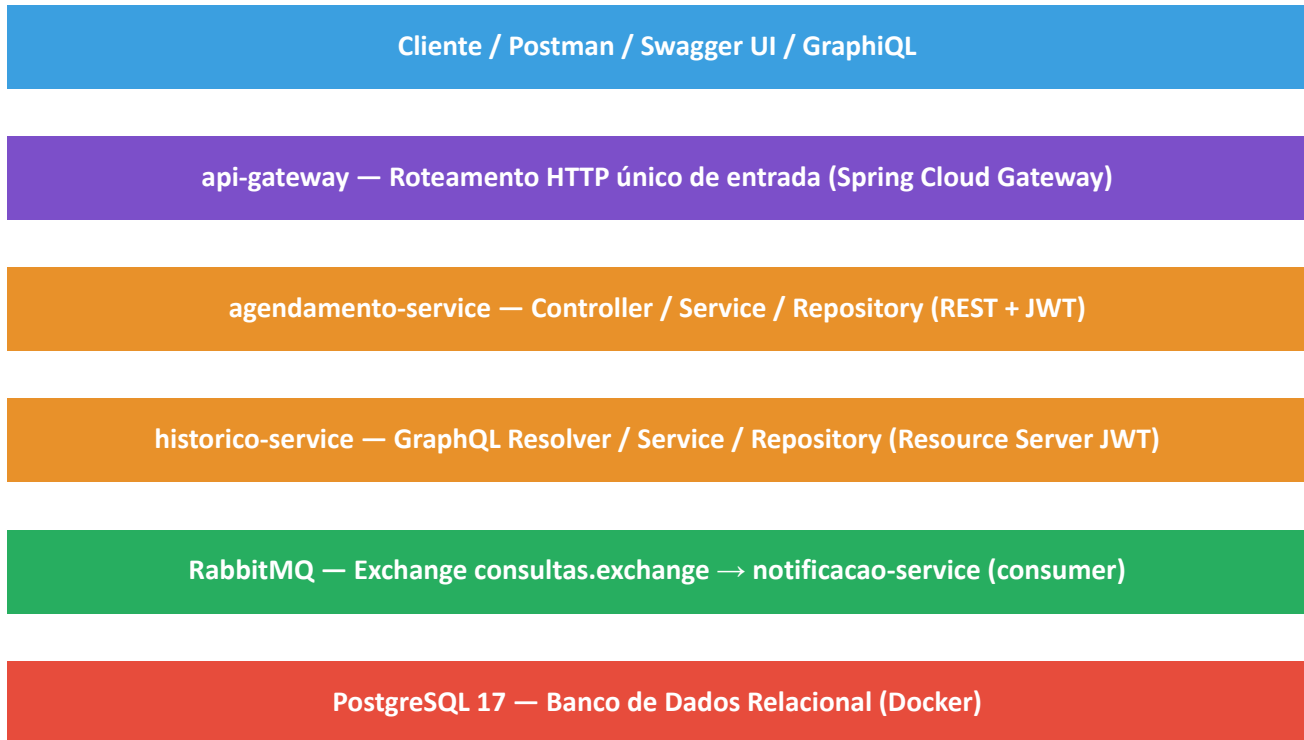


Tabela de Serviços

Serviço	Porta	Responsabilidade	Persistência
api-gateway	8080	Roteamento HTTP único de entrada	—
agendamento-service	8081	Cadastro de usuários, autenticação (JWT), CRUD de consultas, publisher RabbitMQ	PostgreSQL
historico-service	8082	Histórico médico via GraphQL, resource server JWT	PostgreSQL (leitura)
notificacao-service	8083	Consumer RabbitMQ, envio (simulado) de lembretes	—

Responsabilidade de cada camada (por serviço)

Camada	Classe	Responsabilidade
Controller	ConsultaController, UsuarioController, AuthController	Recebe requisições HTTP, valida entrada com @Valid e delega ao Service
Service	ConsultaService, UsuarioService, AuthService, JwtService	Regras de negócio, emissão/validação de JWT, controle de acesso por perfil

Repository	ConsultaRepository, UsuarioRepository	Interfaces Spring Data JPA para acesso ao banco
Model	Usuario, Medico, Enfermeiro, Paciente, Consulta	Entidades JPA com herança JOINED
DTO	ConsultaRequest, ConsultaResponse, CreateUsuarioRequest etc.	Desacopla a API do modelo interno e valida entrada
GraphQL	HistoricoGraphQLController, HistoricoService	Resolve queries de histórico com checagem de acesso por perfil
Messaging	RabbitMQConfig, ConsultaEventConsumer	Publica e consome eventos de consulta via RabbitMQ

2. Segurança e Perfis de Acesso

A autenticação é feita via JWT (HS256). O agendamento-service atua como emissor do token (endpoint de login), validando login e senha com BCrypt. O historico-service atua como resource server OAuth2, validando o mesmo token de forma stateless — nenhum dos dois serviços mantém sessão em memória.

Cada usuário possui um perfil (MEDICO, ENFERMEIRO ou PACIENTE), modelado como subclasse JPA de Usuario. O perfil é embutido no token como claim e traduzido em uma authority do Spring Security (ROLE_MEDICO, ROLE_ENFERMEIRO, ROLE_PACIENTE), utilizada em @PreAuthorize nos controllers REST e nos resolvers GraphQL.

Matriz de Permissões

Ação	Médico	Enfermeiro	Paciente
Cadastrar consulta	Sim	Sim	Não
Editar consulta	Sim	Não	Não
Ver todas as consultas	Sim	Sim	Não
Ver consulta por ID / histórico de paciente	Sim	Sim	Apenas as próprias
GraphQL: histórico por médico	Sim	Sim	Não
GraphQL: histórico / consultas futuras do paciente	Sim	Sim	Apenas as próprias

A restrição "paciente só vê as próprias consultas" é reforçada na camada de serviço, comparando o identificador do usuário autenticado com o patientId solicitado — tanto no agendamento-service (REST) quanto no historico-service (GraphQL).

Correção de segurança aplicada durante a validação

Durante os testes de ponta a ponta, foi identificado que o subject do JWT carrega o login do usuário (string), enquanto o historico-service assumia incorretamente que esse valor era o ID numérico do paciente, causando NumberFormatException em consultas GraphQL feitas por um paciente. A correção adicionou um claim dedicado (userId) ao token, consumido pelo historico-service através de um JwtAuthenticationConverter configurado para usar esse claim como principal — mantendo o subject (login) intacto para a lógica já existente do agendamento-service.

3. Modelagem de Domínio

O sistema utiliza herança de entidades JPA com estratégia JOINED para os usuários: os campos comuns ficam na tabela usuarios, e cada perfil possui sua própria tabela filha com chave estrangeira referenciando a tabela pai.

Hierarquia de Classes

```
Usuario (abstract) <- campos comuns: id, nome, email, login, senha, lastModifiedAt
| -- Medico          -> tabela: medicos
| -- Enfermeiro      -> tabela: enfermeiros
| -- Paciente        -> tabela: pacientes
```

Campos da tabela usuarios

Campo Java	Coluna SQL	Tipo	Restrições
id	id	BIGINT	PK, IDENTITY
nome	nome	VARCHAR	NOT NULL
email	email	VARCHAR	NOT NULL, UNIQUE
login	login	VARCHAR	NOT NULL, UNIQUE
senha	senha	VARCHAR	NOT NULL (BCrypt)
lastModifiedAt	last_modified_at	TIMESTAMP	—

Entidade Consulta

Campo Java	Coluna SQL	Tipo	Restrições
id	id	BIGINT	PK, IDENTITY
pacienteId	paciente_id	BIGINT	NOT NULL
medicoId	medico_id	BIGINT	NOT NULL
registradoPorId	registrado_por_id	BIGINT	NOT NULL
dataHora	data_hora	TIMESTAMP	NOT NULL
status	status	VARCHAR	NOT NULL (AGENDADA, REALIZADA, CANCELADA)
observacoes	observacoes	VARCHAR(1000)	—
criadoEm	criado_em	TIMESTAMP	NOT NULL, imutável
lastModifiedAt	last_modified_at	TIMESTAMP	—

4. Endpoints Disponíveis (REST)

Todos os endpoints REST seguem o padrão versionado sob o prefixo /api/v1, expostos diretamente pelo agendamento-service (porta 8081) ou através do api-gateway (porta 8080).

Método	URL	Perfis / Acesso	Descrição
POST	/api/v1/usuarios	Público	Cadastrar usuário (MEDICO, ENFERMEIRO ou PACIENTE)
POST	/api/v1/auth/login	Público	Autentica e retorna o token JWT
POST	/api/v1/consultas	MEDICO, ENFERMEIRO	Agendar consulta
PUT	/api/v1/consultas/{id}	MEDICO	Atualizar data/status/observações
GET	/api/v1/consultas/{id}	MEDICO, ENFERMEIRO, PACIENTE*	Buscar consulta por ID
GET	/api/v1/consultas	MEDICO, ENFERMEIRO	Listar todas as consultas
GET	/api/v1/consultas/paciente/{pacientId}	MEDICO, ENFERMEIRO, PACIENTE*	Histórico de um paciente

* Paciente só acessa os próprios dados.

Exemplo 1 — POST /api/v1/usuarios (Cadastrar Usuário)

Requisição:

```
{
  "nome": "Ana Souza",
  "email": "ana.souza@hospital.com",
  "login": "ana.souza",
  "password": "senha123",
  "tipoUsuario": "MEDICO"
}
```

Resposta de sucesso (201 Created):

```
{
  "id": 1,
  "nome": "Ana Souza",
  "email": "ana.souza@hospital.com",
  "login": "ana.souza",
  "tipoUsuario": "Medico"
}
```

Exemplo 2 — POST /api/v1/auth/login

Requisição:

```
{
  "login": "ana.souza",
  "password": "senha123"
}
```

Resposta de sucesso (200 OK):

```
{
  "token": "eyJhbGciOiJIUzI1NiJ9. ...."
}
```

Exemplo 3 — POST /api/v1/consultas (Agendar Consulta)

Requisição:

```
{
  "pacienteId": 1,
  "medicoId": 2,
  "dataHora": "2026-10-01T14:30:00",
  "observacoes": "Retorno pós-cirúrgico"
}
```

Resposta de erro — acesso negado por perfil (403 Forbidden):

```
{
  "status": 403,
  "error": "Forbidden",
  "message": "Acesso negado"
}
```

5. GraphQL — Histórico do Paciente

Endpoint: POST /graphql, servido pelo historico-service (roteável via api-gateway). Playground interativo disponível em /graphiql.

Schema principal

```
type Query {
  historicoPaciente(pacienteId: ID!): [Consulta!]! # MEDICO, ENFERMEIRO, PACIENTE (proprio)
  historicoMedico(medicoId: ID!): [Consulta!]!    # MEDICO, ENFERMEIRO
  consultasFuturas(pacienteId: ID!): [Consulta!]! # PACIENTE (proprio)
  consulta(id: ID!): Consulta                     # MEDICO, ENFERMEIRO, PACIENTE
}
```

Exemplo de query

```
query {
  historicoPaciente(pacienteId: "1") {
    id
    dataHora
    status
    diagnostico
  }
}
```

Queries e perfis permitidos

Query	Perfis permitidos
historicoPaciente	MEDICO, ENFERMEIRO, PACIENTE (apenas o próprio)
historicoMedico	MEDICO, ENFERMEIRO
consultasFuturas	PACIENTE (apenas o próprio)
consulta	MEDICO, ENFERMEIRO, PACIENTE

6. Comunicação Assíncrona (RabbitMQ)

A comunicação entre agendamento-service e notificacao-service é assíncrona, via RabbitMQ, utilizando um exchange do tipo topic e duas routing keys distintas para os eventos de criação e atualização de consulta.

Elemento	Valor
Exchange	consultas.exchange (topic)
Routing key — criação	consulta.criada
Routing key — atualização	consulta.atualizada
Fila (notificacao-service)	notificacoes.queue
Producer	agendamento-service (RabbitTemplate, Jackson2JsonMessageConverter)
Consumer	notificacao-service (@RabbitListener)
Management UI	http://localhost:15672 (guest / guest)

Fluxo validado em ambiente real (containers Docker): ao criar ou atualizar uma consulta pela API, o evento é publicado no exchange, roteado para a fila de notificações e consumido pelo notificacao-service, que registra o lembrete correspondente — confirmado nos logs do serviço durante a validação end-to-end.

7. Documentação Swagger / OpenAPI

A documentação do agendamento-service é gerada automaticamente pela biblioteca SpringDoc OpenAPI, com enriquecimento via anotações @Operation e @Tag.

Swagger UI disponível em: <http://localhost:8081/swagger-ui/index.html>

O que foi documentado

- Endpoints de autenticação, usuários e consultas com @Operation (summary e description)
- Agrupamento por @Tag: Autenticação, Usuários, Consultas
- Validação de campos obrigatórios via Bean Validation (@Valid) refletida no schema
- Perfis aceitos em tipoUsuario: MEDICO, ENFERMEIRO, PACIENTE

8. Coleção Postman

A coleção foi exportada como clinic-scheduler-postman-collection.json (Collection v2.1), com a variável de ambiente {{baseUrl}} configurada como http://localhost:8080 (api-gateway). Está organizada em 6 pastas, cobrindo o fluxo completo da aplicação.

Pasta	Conteúdo
1 - Usuários	Cadastro de médico, enfermeiro e paciente
2 - Autenticação	Login de cada perfil
3 - Consultas	Criar, listar, buscar por ID, histórico por paciente, atualizar
4 - Segurança / Erros	Casos negativos: acesso sem token (401), ações fora do perfil (403)
5 - Histórico GraphQL	Query historicoPaciente para médico, enfermeiro e paciente
6 - Fluxo de Integração	Criação de consulta para validar o disparo da notificação assíncrona

Resultado da execução (Newman, contra o ambiente em Docker)

Cenário	Método	Status Esperado
Cadastro de usuário — Médico/Enfermeiro/Paciente	POST	201 Created
Login — Médico/Enfermeiro/Paciente	POST	200 OK
Criar consulta — Médico/Enfermeiro	POST	201 Created
Listar / buscar consultas	GET	200 OK
Atualizar consulta — Médico	PUT	200 OK
Listar consultas sem token	GET	401 Unauthorized
Criar consulta como Paciente	POST	403 Forbidden
Atualizar consulta como Enfermeiro	PUT	403 Forbidden
GraphQL sem token	POST	401 Unauthorized
GraphQL histórico do paciente — 3 perfis	POST	200 OK

21/21 Requests

24/24 Assertions

0 Falhas

100% Sucesso

A primeira execução revelou o bug de segurança descrito na Seção 2 (NumberFormatException nas queries GraphQL feitas por pacientes). Após a correção, a suíte completa passou a executar sem falhas, e os logs do notificacao-service confirmaram o consumo dos eventos assíncronos gerados pelas consultas criadas durante o teste.

9. Estrutura do Banco de Dados

Banco utilizado: PostgreSQL 17, executando em container Docker, compartilhado entre agendamento-service (escrita) e historico-service (leitura). As tabelas de usuário seguem a estratégia de herança JOINED.

Tabela: usuarios (+ tabelas filhas medicos / enfermeiros / pacientes)

Coluna	Tipo SQL	Restrições
id	BIGINT	PK, IDENTITY
nome	VARCHAR(255)	NOT NULL
email	VARCHAR(255)	NOT NULL, UNIQUE
login	VARCHAR(255)	NOT NULL, UNIQUE
senha	VARCHAR(255)	NOT NULL
last_modified_at	TIMESTAMP	—

Tabela: consultas

Coluna	Tipo SQL	Restrições
id	BIGINT	PK, IDENTITY
paciente_id	BIGINT	NOT NULL
medico_id	BIGINT	NOT NULL
registrado_por_id	BIGINT	NOT NULL
data_hora	TIMESTAMP	NOT NULL
status	VARCHAR	NOT NULL (AGENDADA, REALIZADA, CANCELADA)
observacoes	VARCHAR(1000)	—
criado_em	TIMESTAMP	NOT NULL
last_modified_at	TIMESTAMP	—

10. Execução com Docker Compose

Cada um dos quatro serviços possui um Dockerfile multi-stage (build com Maven Wrapper + JDK 21, runtime com JRE 21). O docker-compose.yml na raiz orquestra toda a stack — PostgreSQL, RabbitMQ e os quatro serviços Java — com healthchecks e variáveis de ambiente para resolução de hosts pelo nome do container na rede interna do Compose.

Variáveis de Ambiente (containers)

Variável	Valor	Descrição
DB_HOST	postgres	Host do PostgreSQL na rede do Compose
RABBITMQ_HOST	rabbitmq	Host do broker RabbitMQ na rede do Compose
AGENDAMENTO_SERVICE_URL	http://agendamento-service:8081	URL usada pelo api-gateway para rotear REST
HISTORICO_SERVICE_URL	http://historico-service:8082	URL usada pelo api-gateway para rotear GraphQL

Passo a Passo para Execução

#	Etapas	Comando / Ação
1	Pré-requisito	Ter Docker e Docker Compose instalados
2	Clonar o repositório	git clone <url-do-repositorio>
3	Entrar na pasta	cd clinic-scheduler
4	Subir os containers	docker compose up --build
5	Verificar containers	docker compose ps
6	Acessar a API	http://localhost:8080/api/v1/usuarios
7	Acessar Swagger	http://localhost:8081/swagger-ui/index.html
8	Acessar GraphQL	http://localhost:8082/graphql
9	Parar os containers	docker compose down

Validação realizada: build e subida executados com sucesso (docker compose up --build), com os 6 containers (postgres, rabbitmq e os 4 serviços Java) iniciando saudáveis e sem erros nos logs de inicialização.

11. Testes Automatizados

Todos os quatro serviços possuem suíte de testes automatizados com JUnit 5 e Mockito, cobrindo controllers, services, filtros de segurança e configuração de mensageria.

Serviço	Cobertura de testes
agendamento-service	AuthController, ConsultaController, UsuarioController, JwtFilter, JwtService, AuthService, ConsultaService, UsuarioService, GlobalExceptionHandler (10 classes de teste + JaCoCo)
historico-service	JwtConfig, SecurityConfig, HistoricoGraphQLController (unitário + integração), ConsultaRepository, HistoricoService
notificacao-service	RabbitMQConfig, ConsultaEventConsumer (unitário e integração com Testcontainers), NotificacaoService
api-gateway	Contexto da aplicação e resolução de rotas (ApiGatewayRoutesTest)

Execução: `cd <servico> && ./mvnw test`

4 Serviços	Todos c/ testes	0 Falhando	100% Verde
------------	-----------------	------------	------------

12. Entregáveis da Fase 3 — Checklist

Requisito do desafio	Status	Observação
Autenticação com Spring Security	Atendido	JWT (HS256), emitido pelo agendamento-service
Níveis de acesso (médico/enfermeiro/paciente)	Atendido	Roles + @PreAuthorize em REST e GraphQL
GraphQL para histórico do paciente	Atendido	historico-service, schema com 4 queries
Serviço de agendamento	Atendido	agendamento-service
Serviço de notificações	Atendido	notificacao-service (consumer RabbitMQ)
Serviço de histórico (opcional)	Atendido	historico-service
Comunicação assíncrona (RabbitMQ/Kafka)	Atendido	RabbitMQ (exchange topic + 1 fila)
Qualidade / boas práticas Spring Boot	Atendido	Arquitetura em camadas, DTOs, exceptions customizadas
Documentação do projeto	Atendido	README.md + este relatório técnico
Collection para testes	Atendido	clinic-scheduler-postman-collection.json, validada via Newman
Repositório de código aberto	Atendido	Repositório Git do projeto
Dockerfile por serviço	Atendido	4 Dockerfiles multi-stage + docker-compose orquestrando tudo

O projeto Clinic Scheduler atende integralmente aos requisitos funcionais e não funcionais definidos no Tech Challenge da Fase 3, com validação end-to-end realizada em ambiente containerizado e correção de uma falha de segurança real identificada durante os testes.